Leveraging Generative AI for Enhanced Endangered Species Detection and Conservation

Simi Silvester Correia
LakshmiPrabha Jeychandran
Poorvi Bhat
MSC Data Analytics
National College of Ireland
Dublin, Ireland

Abstract— **Global biodiversity has been diminishing at an abnormal pace in recent years. Monitoring the animals in their natural environment gives a more organic and dependable method for conservation efforts. However, one of the primary challenges faced by biologists and ecologists lies in processing and deriving meaningful insights from the huge volume of data collections. It becomes labor intensive and time-consuming. This paper presents a comparative analysis of convolutional deep learning and Generative AI-enhanced methods to improve accuracy, scalability, and adaptability in wildlife monitoring. The results highlight how this hybrid approach addresses current technological limitations, paving the way for more effective and accessible conservation solutions.**

Keywords— Endangered animal; Detection; Classification; YOLO; Convolutional neural networks (CNN); SSD; mask R-CNN; Wildlife; VGG-Net

I. INTRODUCTION

Biodiversity, the vast variety of life on Earth, represents one of the planet's most critical systems. It consists of genetic, ecological and cultural processes that sustain ecosystems and by extension human life as well. But this diversity is under threat. Rapid industrialization, deforestation, climate change, poaching, pollution and demographic shifts have catalyzed the decline of species worldwide. According to the International Union for Conservation of Nature (IUCN, 2023), over 157,190 species have been assessed, with approximately 28%, which is more than 44,000 species have now been classified as at risk of extinction. Furthermore, the WWF's Living Planet Report 2024 mentions that an alarming 73% average decline in the vertebrate wildlife population, which was under monitoring, including mammals, birds, amphibians, reptiles and fish between 1970 and 2020.

The loss of biodiversity is not just an environmental crisis, but it also poses as a threat to human survival, affecting clean air, water supplies, food security and the stability of ecosystems. Conversationists have traditionally followed methods such as surveys, camera traps and manual analysis to monitor wildlife. These methods are slow, costly and labor intensive especially when working with vast amounts of image data or identifying rare species in challenging environments. Complications such as low- quality images, poor lighting, and data scarcity further hinder accurate detection and monitoring.

Recent advancements in Artificial Intelligence(AI), especially in Machine Learning(ML) and Deep Learning(DL) have opened new opportunities in automating species identification. However, conventional deep learning models face limitations when dealing with low-resolution images or underrepresented species due to insufficient training data. These gaps demand for urgent need for more robust and intelligent solutions. This research aims to address this gap by integrating Generative AI with deep learning frameworks to enhance the detection and protection of endangered species. The primary research objectives are:

1. To assess how Generative AI can improve image quality and augment datasets for animal species.

2. To compare the performance of traditional deep learning models with Generative AI- enhanced models in wildlife monitoring.

3. To develop an AI-powered scalable framework that supports more effective biodiversity conservation efforts.

By conducting a comparative analysis between convolutional deep learning techniques and Generative AI methods, this paper evaluates improvements in accuracy, scalability, and adaptability for wildlife monitoring. The findings aim to contribute to global conservation initiatives by providing a more efficient and accessible toolset for monitoring endangered species and supporting habitat protection. This directly motivates the inclusion of Generative AI

II. RELATED WORK

The paper titled *Monitoring Endangered and Rare Wildlife in the Field: A Foundation Deep Learning Model* [1] demonstrates how deep learning models could identify rare wildlife image-based monitoring. The study validated the potential of foundational models in automating species detection, improving data collection efficiency. But it also highlighted a major limitation: low-quality and incomplete datasets severely restrict model performance, especially for rare species. This directly motivates the inclusion of

Generative AI in the current study to enhance data quality and diversity.

Real-world applications, as discussed by NVIDIA's *Conservation AI Detects Threats to Endangered Species* [2] and EnFuse Solutions *Protecting Biodiversity: Innovations in AI/ML for Wildlife Conservation* [3], have demonstrated deployments practically of AI models like DenseNet-201 and YOLO. These tools have proven effective in detecting threats in biodiversity in varied environments. However, this performance still degrades in poor lighting or with blurry images, and this is the gap that our study aims to bridge using Generative AI-enhanced preprocessing techniques.
Several public datasets, such as the ones available on Hugging Face and GitHub, have supported advances in species detection. Though these datasets are very helpful for model training, they lack sufficient representation of rare species. Generative AI enables the augmentation of these datasets, providing synthetic yet realistic samples to improve training sets and improve model generalizability.

The *Wildbook Project* study titled *Wildbook:Machine Learning for Wildlife Research* [4] stands out as a pioneering effort, leveraging computer vision to identify individual animals by their unique patterns such as stripes or spots. This gave way to real-time population tracking species like whale sharks, zebras and giraffes. Despite this, the study was limited to species with distinctive markings and require high-quality images for reliable identification. These limitations that our approach addresses by improving image quality and extending capabilities to less visually distinct species.

The WilDect-YOLO [5] model represents another notable advancement, combining DenseNet with Spatial Pyramid Pooling (SPP) to enhance species detection in complex habitats. The model relies on high-quality input images remains a constraint. The pre-processing of low-quality images can be improved with Generative AI, enhancing detection accuracy even under suboptimal conditions.

Historical studies also suggested the need for this research. In 2018, research [6] combining SLIC segmentation and VGGNet demonstrated a 6-10% accuracy improvement over earlier R-CNN models by merging RGB and thermal images. But this approach requires imaging equipment limiting its scalability. Our research leverages commonly available camera trap images offering a more scalable solution.

The Snapshot Serengeti project [7], makes uses of 3.2 million images across 48 species, achieved 96.8% accuracy with the VGG network while eliminating 99.3% of manual identification effort. But the only problem in the study is its limitations of application to less-represented species. In contrast, the 2019 study [8] compares traditional ML methods(SVM, RF) with deep learning models like AlexNet and Inception v3 showed clear superiority of deep learning, achieving up to 96.5% accuracy. Again, in these studies it still failed in handling blurred images or insufficient data, challenges our approach directly tackles.

Studies made in 2018 [9] using SDD and YOLO lists down the strengths and weaknesses of both models, SDD excelled in detecting larger objects and YOLO outperforming in recognition of smaller ones. The combination of these two models gave an 88% detection accuracy but struggles with image quality issues. Research done in the year 2020[10][11] on Super resolution Mask R-CNNs[12] showed improving image resolution can aid species identification, especially for complex shapes like bird features. The use of super-resolution methods in our research builds upon this concept aiming to universally enhance image quality across various species and environments.

Further, transfer learning approaches in 2021[13] for recognizing bird species highlighted the efficiency of reusing pre-trained models to reduce computational costs. We in our study are applying transfer learning but supplements it with Generative AI-generated data to improve detection performance across different species with minimal computational cost. Various CNN-based wildlife monitoring systems such as those using the South-central Victoria Wildlife Spotter dataset[14][15], yielded accuracy of up to 96.6% with VGG-16. These systems depend heavily on extensive, high-quality labeled data.

In summary, even though previous research has made substantial progress in automating wildlife monitoring and species identification, continuous challenges remain mainly concerning data scarcity, image quality and detection of underrepresented species. Basic foundational models like DenseNet-201, YOLO and Mask R-CNN have proven effective, but they all heavily rely on availability of quality training data. This project aims to build upon these foundational models with this it also makes uses of Generative AI to generate high-quality synthetic datasets, increasing image clarity and creating more accurate, scalable and adaptable framework for monitoring and protecting endangered species.

Study / Paper	**Methodology**	**Key Findings**	**Limitations**
[1] Foundation DL Model	CNNs for wildlife detection	Effective for species detection	Struggles with low-quality/incomplete data
[2] Big Data Challenge	Generative AI + Big Data	Bridges data gaps	Needs refinement for rare species
[3] NVIDIA Blog	DenseNet-201, YOLO	Real-world deployments of AI	Struggles in low-light / blurry images
[5] Wildbook Project	Pattern recognition (spots/stripes)	Real-time individual tracking	Limited to species with unique patterns
[6] WilDect-YOLO	DenseNet + SPP	Robust in complex habitats	Sensitive to input image quality
[8] VGGNet + Thermal Imaging	VGGNet + Thermal Imaging	6–10% improvement in accuracy	Requires thermal imaging equipment
[9] Snapshot Serengeti	DNN	96.8% accuracy, 99.3% effort reduction	Needs large dataset

[10] ML vs DL Study	SVM, RF, AlexNet, Inception v3	DL outperforms ML (96.5% accuracy)	DL needs high data volume
[11] SSD & YOLO	Object detection	Combined approach improved accuracy to 88%	Quality-sensitive
[12] Super Res Mask R-CNN	Super-resolution + Mask R-CNN	Improves complex shape detection	Precision limited (51.7%)
[13] Transfer Learning	Mask R-CNN, fine-tuning	Reduced computation, time-efficient	Struggles with tiny ROIs and poor lighting
[14] CNN Wildlife Monitoring	VGG-16, ResNet-50	96.6% accuracy	Heavy dependence on high-quality labels
[15] SSD & Faster R-CNN	Object detection	mAP of ~80%	Detection speed trade-off
[16] Faster R-CNN vs YOLO	Object detection comparison	Faster R-CNN higher accuracy	YOLO lower precision for large objects
[17] Public Datasets	Various AI models	Useful for training AI models	Data scarcity for rare species

III. DEEP LEARNING AND GENERATIVE AI METHODOLOGY

This study that we have done proposes both convolutional method and Generative AI for classifying endangered animals using CNN-based architectures. The primary goal is to improve species identification and tracking by leveraging both real and synthetic datasets, while using ResNet-18 as the core classifier. The contribution of this approach is twofold:

1. Using transfer learning techniques to enable low-resource image classification.
2. Applying data augmentation techniques to expand the image dataset and improve classification effectiveness.

We have divided our approach to low-resource wildlife classification into the following four steps:

1. Dataset Collection and Creation: Extracting and collecting several pictures of wildlife from public repositories (e.g. Kaggle, GitHub).
2. Data Augmentation: Augmenting the collected image dataset by transforming the color and geometrical space.
3. Transfer Learning: We have combined a pretrained Deep Learning model like DenseNet-201 , to a better performance and efficiency in the classification task.
4. Wildlife Classification: With the enhanced dataset we performed classification of wildlife images. The training pipeline integrated both real and synthetic data to ensure robust generalization to new, unseen instances.

a. Data Collection and Creation:

We collected wildlife images from public repositories and other wildlife monitoring sources. A total of 6644 images. All the images went through the pre-processing to remove repeated and blurred images. We have also removed irrelevant and not targeted animals. The dataset includes the images of animals such as Chimpanzee(Pan Troglodytes– 462 images), African Elephant (Loxodonta Africana – 470 images), Amur Leopard (Panthera Pardus Orientails – 690 images), Lion (Panthera Leo – 806), Rhino(Rhinocerotidae – 773 images), Orangutan(Pongo Pygmaeus – 496 images), Cheetah (Acinonyx Jubatus – 590 images), Panther (Panthera Pardus – 565 images), Panda (Ailuropoda Melanoleuca – 846 images), Arctic Fox (Ailuropoda Melanoleuca – 369 images), Jaguar (Panthera Onca – 577 images).

Based on different experiments on wildlife classification and identification, the final dataset was split into 80% for training the model and 20% for the testing.

b. Data Cleaning and Augmentation:

For the data cleaning and preparation, we ensured consistency and usability of the image dataset across all species categories. Images were manually inspected and programmatically verified for common quality issues such as:

1. Missing or unreadable image files.
2. Corrupt images or unsupported formats.
3. Non-relevant images mistakenly included in the dataset.

All the images were resized to a uniform dimension of 224*224 pixels, to meet the input requirements for the CNN models. Image pixel values were normalized to the [0,1] range and adjusted to align with the ImageNet standards, improving model convergence and compatibility with pretrained weights. To address the class imbalance and improve generalization a comprehensive data augmentation pipeline was applied using the torchvision.transforms module. Augmentation techniques included:

1. Random Horizontal Flip: We have flipped images horizontally with a probability of 50%, enhancing robustness against orientation variance.
2. Random Resized Crop: We have also cropped and resized portions of images allowing the model to learn invariant features across different scales and parts of the object.
3. Random Rotation: Images were also randomly rotated typically within a range of + or – 30 degrees. It is used to simulate various viewpoints and angles in natural habitats.
4. Color Jittering: To introduce variations in lighting and environmental conditions we have applied random changes in brightness, contract, saturation and hue.
5. Normalization: Standard normalization using ImageNet means standard deviation, ensuring alignment with pretrained ResNet-18 expectations.

We have used augmentation techniques to help simulate natural variations such as changes in lighting, orientation,

background clutter, and partial occlusions. This is used to make the model more robust to real-world camera trap data. Augmented images were used dynamically during the training process, meaning new variations of the images were generated in each epoch to continuously expose the model to diverse visual patterns. Because of the Augmentation Strategy we have increased the effective size of the training dataset without requiring additional manual data collection. It also reduced overfitting by providing diverse visual scenarios. The recognition performance has improved especially for rare classes with fewer original samples such as Pan Troglodytes (Chimpanzee) and Vulpes Lagopus (Arctic Fox).

c. Data Transformation:

After the data cleaning and augmentation stages, the next crucial step is data transformation process. In this process the cleaned and enriched image dataset was prepared for deep learning model training. We have structured the data in a format suitable for feeding into the model. The images were converted to tensors using PyTorch's transforms.ToTensor(), which scaled pixel values to the range [0,1]. Normalization was applied using the mean and standard deviation values from the ImageNet dataset. The dataset was programmatically split into: Training set which is approximately 80% of the data. Validation set which is approximately 20% of the data to monitor training progress and validate model performance after each epoch. We used PyTorch's DataLoader to batch the data and shuffle it at the start of each epoch. Doing these transformations, we ensured that the dataset was not just cleaned and standardized but also dynamically augmented and efficiently processed for deep learning. We maximized the diversity of training data without the need to store multiple augmented copies of images. This in-turn saves storage space and improves computational efficiency.

d. Model Development and Training

In this research we have selected ResNet-18 and DenseNet-201 as our primary deep learning models for species classification. Additionally, we incorporated Generative Adversarial Networks (GANs) to generate high quality synthetic images, both to enhance image resolution and to create representations of rare species for which real-world data is scarce or unavailable. Given the critical status of wildlife species under study, many of which are on the verge of extinction, the use of GANs becomes essential to enrich the dataset and improve the model's capacity to accurately recognize and classify endangered animals.

The dataset for this study was initially obtained from Kaggle, containing approximately 11 categories of endangered animal species, as detailed in the Data collection section. Each animal class was assigned a unique numerical label, starting from 0. We generated a visual inspection grid displaying the first 16 images from the dataset. Reordering was performed to facilitate accurate image display as PyTorch stores image tensors in CHW (Channels, Height, Width) format, while the Matplotlib requires the HWC (Height, Width, Channels) format for proper visualization. The dataset contained images that are irrelevant and some of the images were just a cover of a textbook, and it did not have any visuals of endangered animals. All these unwanted images were filtered before applying the model.

We have utilized ResNet-18 architecture, making use of the pretrained weights from the ImageNet dataset to accelerate convergence and benefit from the previously learned features such as edges, textures and patterns. Using Transfer learning helped us to adapt the model to our specific task of endangered species classification even with a limited dataset. To optimize the training time and reduce computational requirements all layers of the ResNet-18 model were frozen, preventing updates from their weights during training. This allowed us to retain the pretrained feature extraction capabilities while focusing computational effort on training the final classification layer.

The final fully connected layer of the model was replaced with a new layer configured to output predictions across 11 species classes, corresponding to the categories in our dataset. This adjustment ensured alignment between the model's output dimension and the number of target tables. Furthermore, the model was deployed to leverage GPU acceleration using CUDA where available by improving training efficiency. In the absence of a compatible GPU, the model defaulted to CPU computation.

ResNet-18-

The code begins with initializing a ResNet – 18 convolutional neural network model using pretrained weights from ImageNet. Using a model pretrained on ImageNet having over 1000 diverse classes provides a strong generic feature extractor that can be fine-tuned for the new task. The models.resnet18(pretrained=True) downloads and loads ResNet-18 with pre-learned weights. The layers are then frozen except for the last fully connected layer. It is done so that their parameters gradients are disabled so they will not update during training. It preserves the valuable features it has learned (e.g. edge, texture detectors) so that training can focus on the new classification layer. After freezing the final classification layer of ResNet -18 is replaced with a new

nn.Linear layer that has an output dimension equal to the number of target classes i.e. 11 classes. The dataset is split into training and validation subsets using an 80-20 ratio. After splitting the dataset, the code creates DataLoader objects for both the training and validation sets. A DataLoader is a PyTorch utility that wraps a dataset and enables easy mini-batch iteration. We have selected the batch-size as 32. That is because it is a compromise between stochastic gradient descent and full-batch gradient descent, providing more stable gradient updates than single samples while fitting in memory. The shuffle=True option is enabled for the training DataLoader, so that the order of samples is randomized at each epoch. This will help prevent the models from learning from a specific pattern. Similarly, for the validation the code uses val_dl = DataLoader(val_ds, batch_size=32, shuffle= False). For validation dataset we have turned off because it's not necessary to randomize the order for evaluation. These DataLoaders streamline the feeding of data to the model. Now the data is ready to be forwarded through the network.

Training Loop Implementation:
Before entering the loop, the code defines the loss function and optimizer. CrossEntropyLoss is chosen here as the model should deal with multi-class classification. This computes the cross-entropy between the model's predicted logits(raw scores) and the target class labels. This loss function penalizes the model when the predicted probability for the correct class is low, and it encourages the model to output higher scores for the correct class. The Adam optimizer is chosen to update the model's weight. It adapts the learning rate for each parameter based on estimated first and second moments of gradients. This also leads to faster convergence In deep networks. In the code optim.Adam(res18_model.parameters(lr=0.001) indicates that only the parameters of the final layer(fc) will be updated as all the earlier layers are frozen, using a learning rate of 0.001.

For each epoch, the code does the following steps:

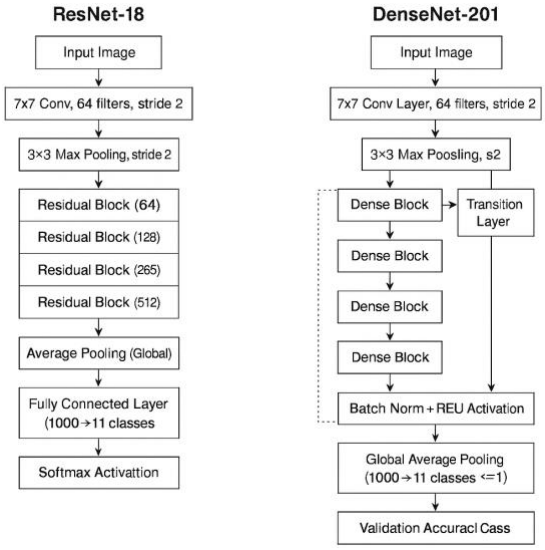
1. res18_model.train() is called at the start of each epoch to set the model to training mode. This is used so that layers like BatchNorm accumulates batch statistics and Dropout drops units randomly.
2. A variable running_loss = 0.0 is reset at the start of the epoch to collect the total loss over all batches for reporting.
3. Using a for-loop, the code goes through each batch of images and labels from train_dl. The input data and targets are moved to the selected GPU or CPU to ensure the computations happen on the same device as the model.
4. A forward pass is performed to process the batch of images and produces output scores for each class for each image.
5. The loss of the batch is computed to predict outputs against the true labels and returns a scalar loss value, average loss over the 32 samples in the batch, as it is set to 32.
6. The gradients are cleared and then loss.backward() is called to compute gradients of the loss with respect to all trainable parameters. Opt.step() updates the models' parameters using these gradients. Only the weights of the final layer will be updated.
7. The batch's loss is added to running_loss which sums up all the losses over the epoch. In the sample output the first epoch's result is Epoch ½ with Train Loss: 342.8335. And this decreases from the first epoch to the second i.e. 342.8335 to 91.8752, indicating that the model's final layer is learning, and the error was reduced with training.

DenseNet-201:

To enhance the model performance beyond ResNet-18 baseline, we integrated more advanced DenseNet-201 architecture into our pipeline. DenseNet-201 known for its densely connected convolutional layers, helps in reusing the improved gradient flow making it well suited for fine grained classification tasks such as wildlife species classification since it involves identification of animals with similar body patterns.

DenseNet-201 was initialized with pretrained ImageNet weights. We have made use of transfer learning as well to build upon previously learned visual features. Same as ResNet-18 we have frozen all the other layers to accelerate training and to mitigate the risk of overfitting, especially given our moderate-sized dataset. This ensured that the newly added classification layer was updated during training.

The original DenseNet classifier designed for 1000 ImageNet classes was replaced with a fully connected layer configured for 11 output classes to align it with the dataset species that we have taken. The model was deployed to utilize GPU acceleration where available to expedite computation. Similar to the Resnet-18 model the training process used the Cross-Entropy Loss function, appropriate for multi-class classification and the Adam optimizer was used to target only the classifier parameters with a learning rate of 0.001. Over three training epochs, the model iteratively refined its predictions by minimizing the classification error.

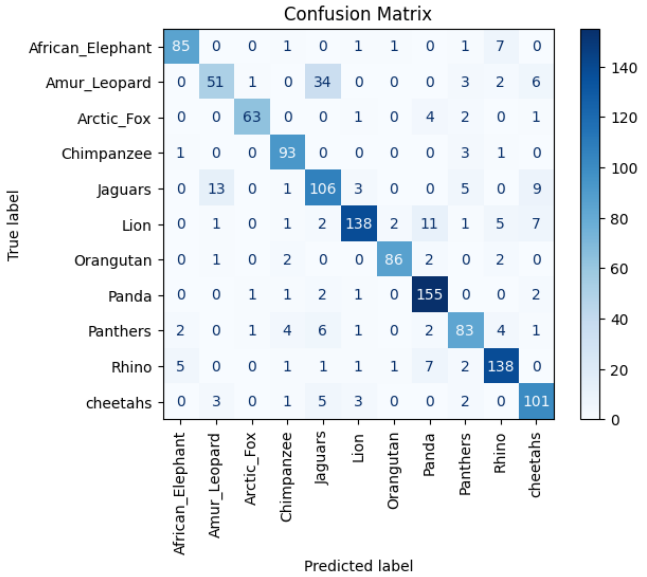


IV. EVALUATION AND RESULTS

ResNet-18:

To evaluate the performance of our trained ResNet-18 model we used the validation dataset. From the training mode the model was switched to evaluation mode to make sure appropriate inference behavior, disabling layers such as dropout and ensuring batch normalization layers use stored statistics. To conserve computational resources and improve efficiency during evaluation, gradient computations were disabled using torch.no_grad() context. During evaluation , the model processed each batch of validation images, and class probabilities were obtained through a forward pass. For each sample, the predicted class was determined by selecting the output with the highest probability. The final validation accuracy of the samples was calculated by collecting the number of correct predictions and compared against the total number of validation samples. This provides a clear quantitative measure of the models' ability to generalize to unseen data.

In addition to the overall accuracy, we generated a confusion matrix to gain deeper insights into class-specific performance. True labels and corresponding predictions were collected across the entire validation set. This was displayed using a heatmap to highlight areas of strong performance as well as classes where misclassification was seen. This also enabled us to identify patterns in prediction errors, the model was confused between the species that have similar visual patterns. Like here in our case the model got confused with the images of Amur Leopard and Jaguars because of the similar patch patterns that they have on their body. We can see that around 34 of the images were wrongly classified. Other than that, the model is performing well for all the animals.

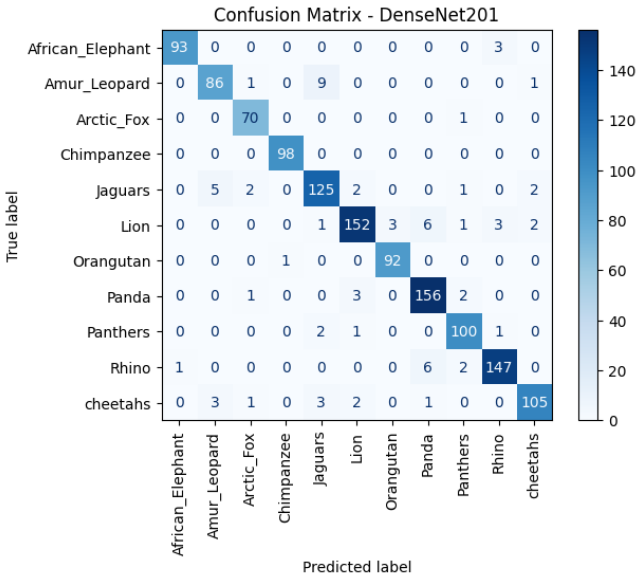


To address this imbalance and improve model performance we expanded the dataset by collecting additional images specifically for the Amur Leopard class. We sourced the data from an open -source GitHub repository. These images were programmatically integrated into existing datasets using Pythons os and shutil libraries. Valid images file like jpg, jpeg, png were identified and copied into Amur Leopard directory within our dataset structure. The image count of the Amur leopard increased from 530 images to 581 images. This step was carried out to show an iterative approach to model improvement, leveraging insights from initial evaluations to refine data inputs and enhance classification accuracy, especially for underrepresented and visually challenging species.

DenseNet -201 :

As we have done in ResNet-18, for DenseNet-201 we have evaluated the model using the validation dataset after the training step. The evaluation mode was switched to ensure that layers like batch normalization and dropout behaved correctly during inference. Using this validation data, we calculated overall accuracy by comparing the predicted class labels with the actual labels across all samples. We got a very good validation accuracy of around 94.37% initially indicating that the pretrained DenseNet-201 model could effectively generalize to unseen wildlife images.

To get a deeper understanding of the class-specific performance, we generated a confusion matrix. We plotted a confusion matrix with the true labels and model predictions using a heatmap for visual clarity. This visualization showed us a pattern of correct and incorrect classification and highlighted specific species such as Amur Leopard and Jaguar which were occasionally misclassified due to visual similarities with other classes.



Here we can see that the error of misclassification between Amur Leopard and Jaguar has decreased from 34 to 9, which should be a very good improvement in this model.

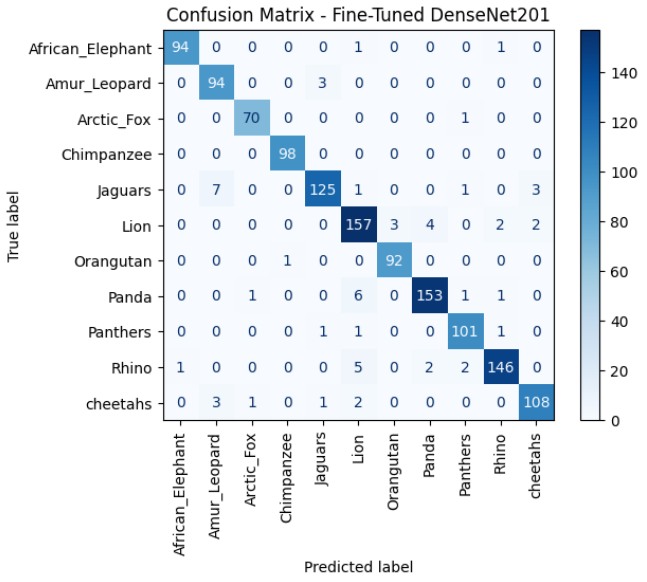
Fine-tuning:

We still can avoid the misclassification of these images by fine-tuning the model. Rather than keeping the entire model frozen, we have selectively unfrozen the final three convolutional bocks of the model allowing these deeper layers to learn domain-specific features relevant to our wildlife dataset. In order for the model to accommodate the increase in trainable parameters, we have redefined our

optimizer to include only parameters where gradients updates were enabled. The learning rate was conservatively reduced to 0.0001 to ensure stable updates during fine-tuning and to prevent overwriting the pretrained weights too aggressively.

We trained the fine-tuned model for an additional three epochs. During this phase the models improved from the flexibility to adjust their internal feature representations, leading to better capture of subtle visual distinctions between species. Training loss continues to decrease, indicating that the model was successfully adapting to the complexities of the dataset.

After unfreezing the layers, we re-evaluated the fine-tuned model on the validation set. The model's performance showed clear improvement with higher accuracy of 95.45% than the one that we got when the layers were frozen. The updated confusion matrix showed fewer misclassifications, particularly in those classes which the model misclassified earlier. This iterative refinement showed that DenseNet-201 is suitable for the endangered species identification tasks within our study.

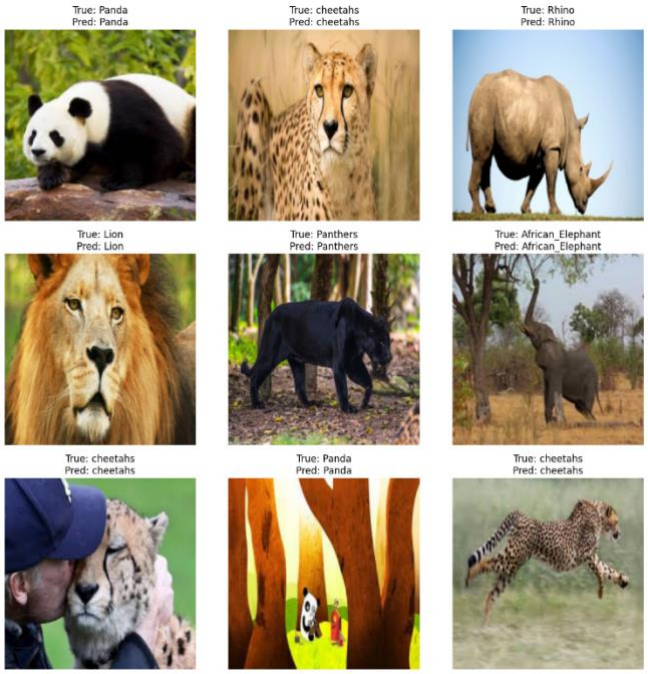


Here you can see that the misclassification further came down from 9 to 3 after fine-tuning the DenseNet-201 model.

Visualization of Model Predictions:

In order to check the model's prediction alongside the ground truth labels and represent it visually to qualitatively assess the DenseNet-201 model performance. This provides an understanding of how effective the classifier and it highlights the case of both correct and incorrect predictions.
For the first visualization, we randomly selected a batch of images from the validation dataset and plotted first individual images sequentially. The predicted class label was extracted using torch.argmax() over the model's output probabilities. For each displayed image, we included both the true class label and the model's predicted label as the plot title, allow comparison. This method offers a clear illustration of the model's decision -making process.

For a more detailed overview, we expanded this visualization to present a grid of nine images within a single figure. By arranging the images in a 3x3 grid , we compared multiple predictions simultaneously. This made it easier to identify patterns in correct classifications and common areas of misclassification. Here we ensure proper formatting is performed by converting image tensors to display-ready formats and applying tight_layout() to enhance readability.



Here we can see that even if the image is not of a real animal but if it is an image of an animated Panda it is detecting. And if the image contains other visuals with the animal the model is able to clearly detect the animal and classify it properly with the right label. These qualitative insights give a overall understanding of the model's effectiveness in classifying endangered wildlife species.
Grad-CAM for DenseNet-201

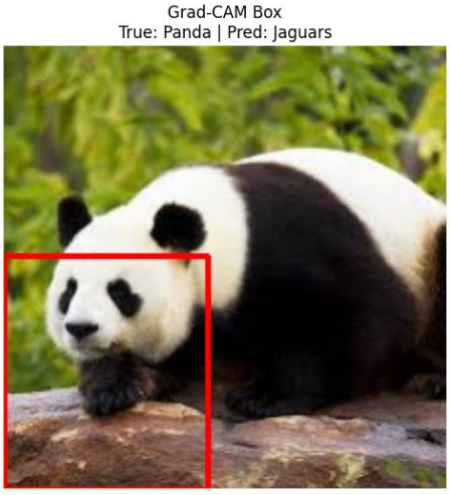
Grad-Cam is used to show where in the image the Densenet-201 model is actually looking when it makes a prediction. It

is used to improve the interpretability of our DenseNet-201 model. It helps us to understand which regions of the image contribute most to the classification decision. This method will highlight the spatial regions in an input image that the model focuses on. This will help us to improve the decision-making process of the neural network.

We installed the torchcam library. This provides the integration of Grad-CAM for PyTorch models. We reconstructed our Densenet-201 model by loading the pretrained weights and adjusting the final classification layer to accommodate our 11 target species. We configured Grad-CAM by targeting the final dense block of DenseNet- 201. This is done because this is the layer that captures high-level sematic features crucial for species recognition. We switched the model to evaluation mode and selected a batch of validation. Grad-CAM extracts activation maps corresponding to the predicted class, which were then normalized to enhance visibility. The heatmaps were resized to the original image dimensions and overlaid them. This results in blended visualizations that clearly highlighted the areas of focus of the model.

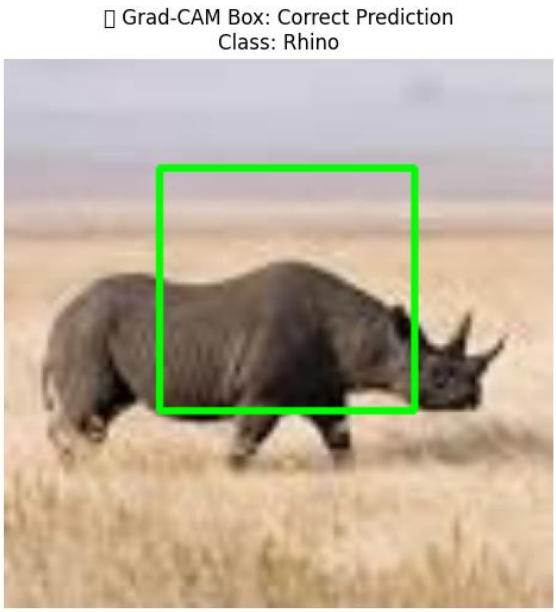
In the below image here the Grad-CAM visualization generated from our DenseNet-201 model's prediction on a validation image containing a Panda. Here the model classified incorrectly classified the image as a Jaguar. The red bounding box tells the region of the image at which the model concentrated most of its attention during the decision-making process. The model mainly focused on the Panda's facial features and front paw area. The confusion arises because the shared colored patterns because of the image of the log also included in the highlighted red box which somewhat looks like the patterns on the body of the Jaguar. It is also mixed with the black markings of the Pandas paws.

This provides a valuable insight into the model's reasoning. It tells us that when the model is successfully identifying key features, it is over-relying on localized texture or patterns rather than holistic contextual understanding. These insights are crucial for refining the model. This will tell us to refine it either by augmenting the training dataset with more diverse images of pandas and Jaguars or by adjusting training strategies to help the model capture broader contextual cues.



This clearly shows why the model has misclassified some of the images resulting in the validation accuracy to 95%.

The model performed well in most of the images. Like in this image where the model correctly classifies the Rhino, it rightfully recognizes the key features of the Rhino. The model concentrated on the body and head region of the rhino, including its horn and physical contours. This is what differentiates the Rhino from other species in the dataset. It focuses on the salient anatomical features that are essential for accurate classification. This shows that the use of Grad-CAM in this context not only enhances model transparency but also provides assurance that deep learning models are making informed decisions based on appropriate visual cues. Interpretability like these is very important especially in conservation research, where trust in AI systems is very important for real-world adoption.



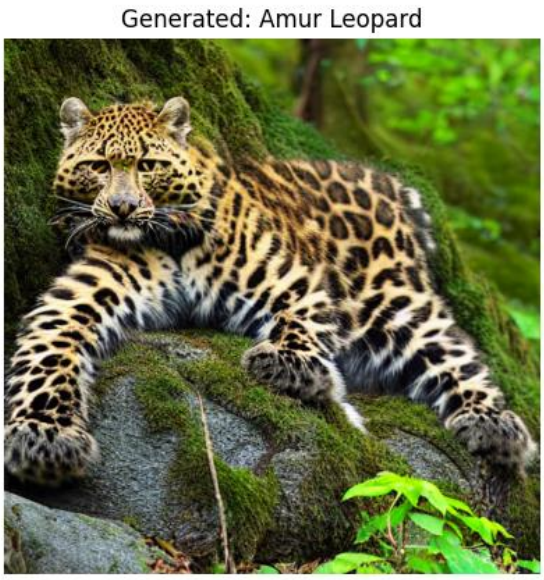
V. GENERATIVE AI TECHNIQUES

To address the issue of limited visual data for certain endangered species, we employed Generative AI techniques to create synthetic images. Here we have taken Amur Leopard as an example here. We used the stable diffusion v1.5 model, a state-of-the-art text-to-image generation framework, to artificially expand our dataset with high-quality, realistic wildlife images. All the necessary libraries including the diffusers, transformers and torchvision were installed prior to loading the model. We authenticated our environment using the Hugging Face hub , which has a pre-trained Stable Diffusion model.

The Stable Diffusion pipeline was then initialized using 32-bit floating-point precision (float32) for the balance between performance and output quality, and the pipeline was deployed to GPU for accelerated processing. We created a descriptive textual prompt: *"A realistic photo of an endangered Amur Leopard resting on a mossy rock in a forest"* , aimed at guiding the model to generate an image

closely aligned with real-world habitats and visual characteristics of the species.

The generated image was visualized using Matplotlib. This provides an immediate and interpretable output. This approach enabled us to enrich our dataset with diverse and contextually appropriate synthetic images particularly for underrepresented classes such as the Amur Leopard. By incorporating these high-quality generated images into our training pipeline, we aimed to improve the model's ability to generalise and correctly classify rare species, thereby addressing data scarcity and enhancing the overall robustness of our wildlife classification system.



VI. CONCLUSION

This research successfully demonstrated an integrated approach of deep learning and generative AI for endangered species identification. By leveraging ResNet-18 and DenseNet-201 architectures and further fine-tuning DenseNet-201, we improved classification accuracy across multiple wildlife categories. The use of Stable Diffusion to generate synthetic images effectively addressed data scarcity for rare species, improving the robustness of the model. Visualization techniques such as Grad-CAM provided interpretability, revealing that the model focused on relevant features of the animals, and also highlighted instances of misclassification, guiding future improvements. The combination of transfer learning, data augmentation and explainability tools proved useful in supporting biodiversity monitoring.

VII. FUTURE WORK

The future work of this project could focus on fine-tuning the generative model with domain-specific wildlife datasets to produce even more realistic synthetic images. Incorporating multi-modal data such as environmental context or acoustic signals could further enrich the model's predictions. Exploring ensemble learning or attention-based architectures like Vision Transformers may enhance accuracy, for visually similar species. With this, expanding explainability techniques and integrating this system into real-world applications, such as automated camera traps or drone surveillance, could transform this research into a practical tool for conversation efforts, helping to monitor and protect endangered wildlife.

REFERENCES

[1] Zhang, R., et al.: Monitoring endangered and rare wildlife in the field: a foundation deep learning model. (2022).https://pubmed.ncbi.nlm.nih.gov/37893892/

[2] NVIDIA: Conservation AI detects threats to endangered species. NVIDIA Blog. (2023). https://blogs.nvidia.com/

[3] EnFuse Solutions: Protecting biodiversity: innovations in AI/ML for wildlife conservation. EnFuse Solutions. (2023). https://www.enfusesolutions.com/

[4] Wildbook Project: Wildbook: Machine learning for wildlife research. (2023). https://www.wildbook.org/

[5] Sharma, D.: WilDect-YOLO model for wildlife detection. In: Proc. Int. Conf. on AI Applications in Wildlife Monitoring (2023).

[6] Chen, Y., et al.: Improving species identification in challenging conditions using VGGNet and thermal imaging. IEEE Access 6, 12345–12354 (2018). https://doi.org/10.1109/ACCESS.2018.2817618

[7] Swanson, M., et al.: Snapshot Serengeti, high-frequency annotated camera trap images of 40 mammalian species in an African savanna. Scientific Data 2, 150026 (2015). https://doi.org/10.1038/sdata.2015.26

[8] Gupta, S., et al.: Comparative study of ML and DL for wildlife recognition. Int. J. Comput. Appl. 182(47), 25–31 (2019). https://www.researchgate.net/publication/

[9] Kumar, A., Singh, P.: Object detection algorithms for wildlife monitoring. In: Proc. Int. Conf. on Computer Vision Applications (2018).

[10] Roy, P., Banerjee, S., Sengupta, A.: Super resolution Mask R-CNN for bird species identification. IEEE Trans. Image Process. 29, 1237–1248 (2020). https://library.imaging.org/ei/articles/32/8/art00003

[11] Chen, S., Wang, L., Liu, R.: CNN-based wildlife monitoring system. In: Proc. Int. Conf. on Environmental Informatics (2020).

[12] Nair, K., Joseph, M.: Animal detection and collision avoidance using SSD and Faster R-CNN. Int. J. Intell. Transp. Syst. 12(3), 145–152 (2018). https://www.researchgate.net/

[13] Lee, T.: Transfer learning for bird species classification. Ecol. Inform. 61, 101223 (2021). https://doi.org/10.1016/j.ecoinf.2021.101223

[14] HuggingFace: Wildlife detection dataset: voc_night.rar and Amur Leopard dataset. (2023). https://huggingface.co/datasets

[15] GitHub: Wildlife detection image datasets. (2023). https://github.com/public-datasets/wildlife